**Module:** **Planning**

DOCUMENTATION

# Path Planning *2*

**Belal** *Elsayed*

**24P0459**

**This Report contains 8 Pages.**

## Main Task

Compute an algorithm that can output a path that can navigate a track by using the colored cones around it and have a failsafe for if the number of cones is unequal on each side.

## How it works

```
        num_points = 25
        step = 0.5
        cx = self.car_pose.x
        cy = self.car_pose.y

        tyaw = self.car_pose.yaw
```

**Initializing the step size for the path generator and the vehicle's current position and heading from its pose.**

```
        def getdist(c):
            return math.hypot(c.x - cx, c.y - cy)
```

**The helper function I will use to sort the cone arrays**

```
        blue_cones = [c for c in self.cones if c.color == 1 and getdist(c) > 1]
        yellow_cones = [c for c in self.cones if c.color == 0 and getdist(c) > 1]

        blue_cones.sort(key=getdist)
        yellow_cones.sort(key=getdist)
```

**I split the cones array by color using the c.color attribute and I ignore cones closer to the rover than 1m as it can cause invalid point creation when the cone is too close to the car, this will be obvious after a few lines:**

```
        points = [(cx,cy)]

        numpairs = min(len(blue_cones), len(yellow_cones))

        #cone pairs
        for i in range(numpairs):
            tx = (blue_cones[i].x + yellow_cones[i].x) / 2.0
            ty = (blue_cones[i].y + yellow_cones[i].y) / 2.0
            points.append((tx, ty))
```

Now I set up the points coordinates array, which will hold the waypoints the car will go through, and we have numpairs, which will store the number of pairs of different colored cones.

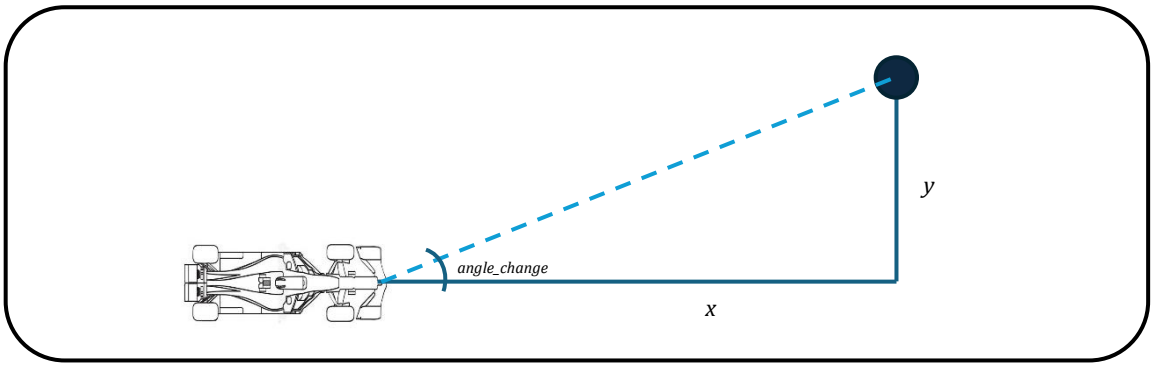
The loop below simply finds the midpoint of these pairs and appends it to the points array.

```
        #blue cones
        for i in range(numpairs, len(blue_cones)):
            lastx, lasty = points[-1]
            angle_change = math.atan2(blue_cones[i].y - lasty, blue_cones[i].x
- lastx)
            tx = blue_cones[i].x + 1.5 * math.cos(angle_change - math.pi / 2)
            ty = blue_cones[i].y + 1.5 * math.sin(angle_change - math.pi / 2)
            points.append((tx, ty))

        #yellow cones
        for i in range(numpairs, len(yellow_cones)):
            lastx, lasty = points[-1]
            angle_change = math.atan2(yellow_cones[i].y - lasty,
yellow_cones[i].x - lastx)
            tx = yellow_cones[i].x + 1.5 * math.cos(angle_change + math.pi /
2)
            ty = yellow_cones[i].y + 1.5 * math.sin(angle_change + math.pi /
2)
            points.append((tx, ty))
```

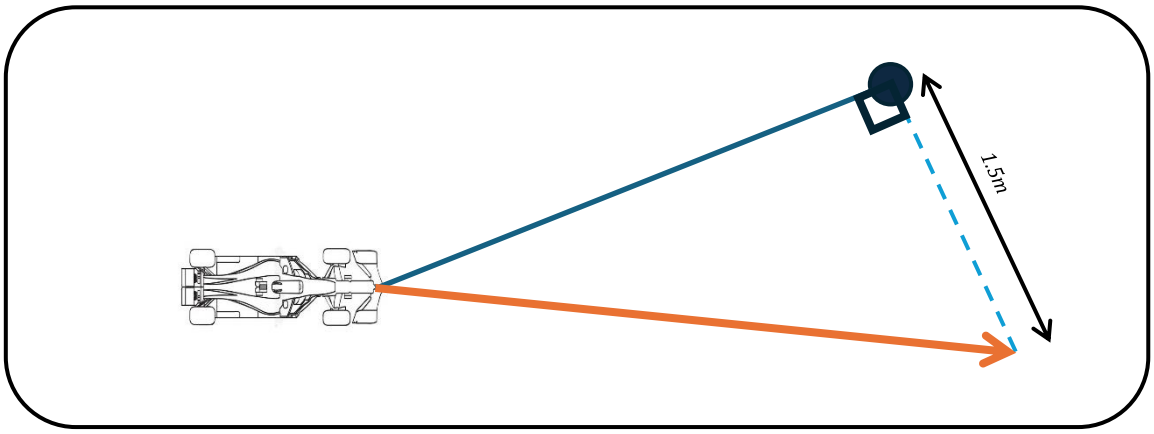
Here, the complexity increased a little, these loops only work if there are remaining unpaired blue/yellow cones. I store the x y data of the last point we stored before, then calculate the angle difference between that cone and the last waypoint.

Then I place a point 1.5m perpendicularly from that cone (to the right of blue cones, left of yellow cones) using cosine and sine functions.




$$tan(angle_change) = \frac{Opposite\ (y)}{Adjacent\ (x)}$$

***Fig 1*** **- How atan2 was used to get the angle difference from the last point to the cone, shows by an extra blue cone**



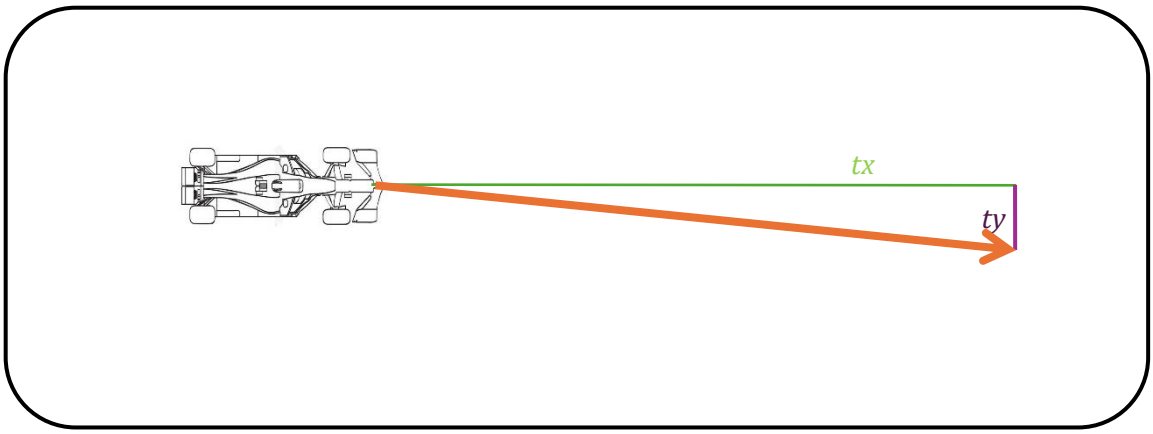




***Fig2,3 - How I used sine and cosine to get tx and ty***

```
        #No cones
        if len(points) == 1:
            tx = cx + 10.0 * math.cos(tyaw)
            ty = cy + 10.0 * math.sin(tyaw)
            points.append((tx, ty))
```

**For a map where no cones are detected, we just get the car to move 10m in the direction it is already heading towards.**

```
        if len(points) >= 2:
            #get the heading of the final segment
            final_yaw = math.atan2(points[-1][1] - points[-2][1], points[-
1][0] - points[-2][0])
        else:
            final_yaw = tyaw

        extension_dist = 6.0
        final_tx = points[-1][0] + extension_dist * math.cos(final_yaw)
        final_ty = points[-1][1] + extension_dist * math.sin(final_yaw)
        points.append((final_tx, final_ty))
```

**Extends the path by finding the slope of the last 2 points and extends it to get a 5-10m line.**

```
        path: Path2D = []

        for i in range(len(points)-1):
            p1 = points[i]
            p2 = points[i+1]
            dist = math.hypot(p2[0] - p1[0], p2[1] - p1[1])

            steps = max(1, int(math.ceil(dist / step)))

            for j in range(steps):
                interpx = p1[0] + (p2[0] - p1[0]) * (j / steps)
                interpy = p1[1] + (p2[1] - p1[1]) * (j / steps)
                path.append((interpx, interpy))

        path.append(points[-1])

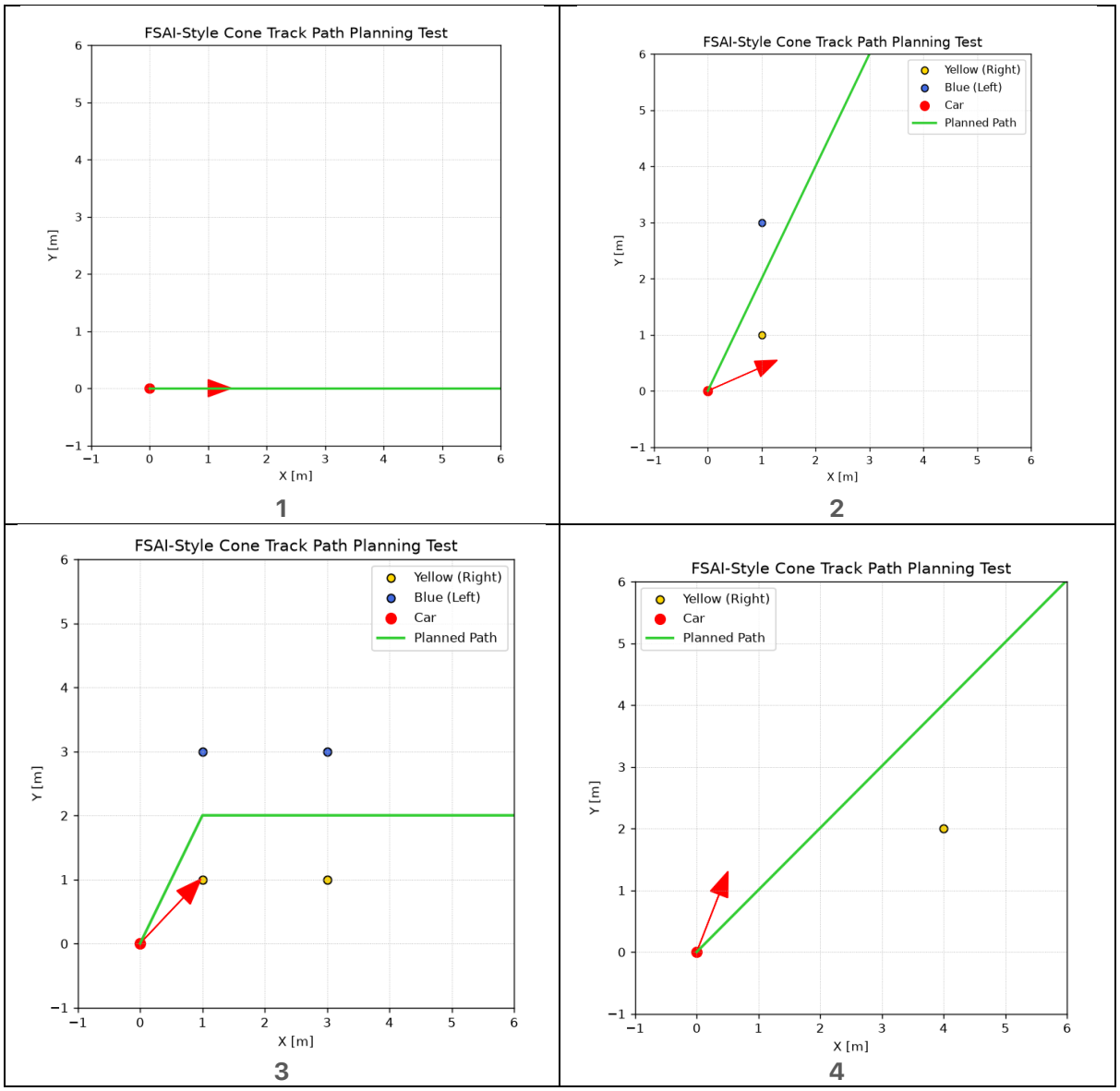
        return path
```

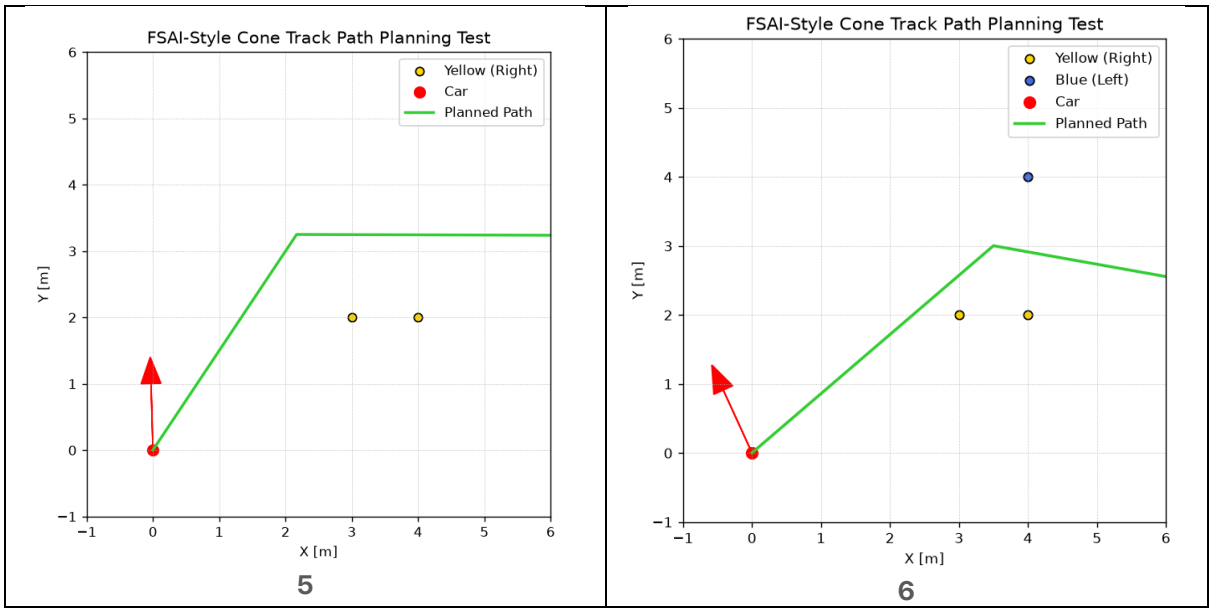
**I create a new list that will contain the final path,**

**The dist.. loop calculates the distance from every waypoint, and fills the gap with 0.5m slices. Interpx and interpy adds a new coordinate point to the map at every slice.**

**Path.append.... adds the final target waypoint to end the path.**

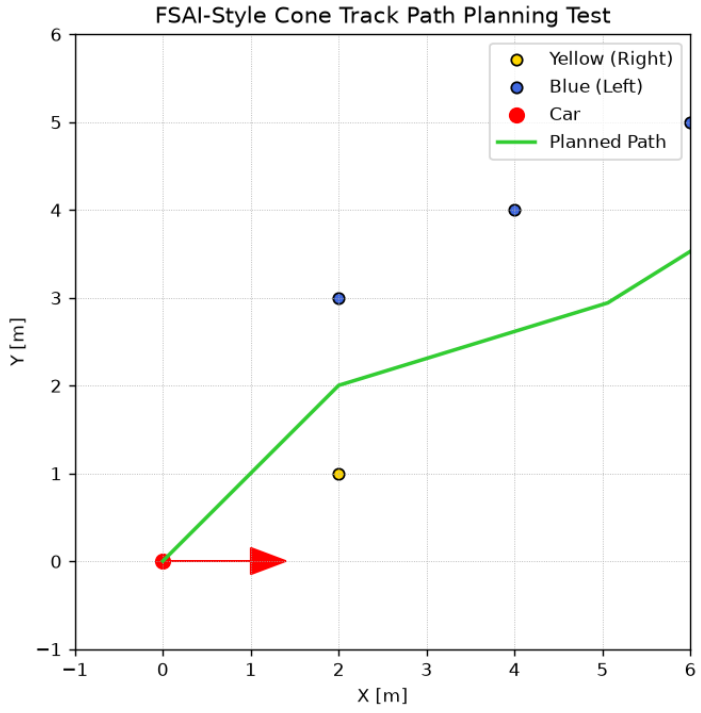
# Scenarios






## Custom Test Case:

## 3 Blue, 1 Yellow



## Why I chose this solution?

It is very versatile, if one side has up to 2 more cones than the left the path finder will still find a path that follows the track limits, however, due to the fact that the pathfinder uses sharp edges and not curves, it can cause the car to stop and turn every corner, which is a problem for FSAI cars.

## Assumptions:

Cones closer than 1m to the car are ignored

The points we estimated from the extra unpaired cones are 1.5m away from it.